Computer Communications 154 (2020) 82–91

Contents lists available at ScienceDirect

Computer Communications

journal homepage: www.elsevier.com/locate/comcom

Poly-stable matching based scalable controller placement with balancing constraints in SDN

Balaprakasa Rao Killi [a,*], Seela Veerabhadreswara Rao [b]

[a] Department of Computer Science and Engineering, National Institute of Technology Warangal, 506004, India
[b] Department of Computer Science and Engineering, Indian Institute of Technology Guwahati, 781039, India

ARTICLE INFO

Keywords:
Software defined networks
Load imbalance
Stable matching
Switch relocation
Controller placement

ABSTRACT

Software Defined Networking achieves programmability of the control plane by separating it from the data plane. In this paper, we propose a strategy for controller placement to minimize the maximum load imbalance between controllers while enforcing the placement and balancing constraints. We also propose a scalable algorithm to compute near optimal solutions of the problem on large scale networks. The algorithm uses poly-stable matching to distribute a fraction of switches equally among controllers so as to reduce the load imbalance. The remaining switches are assigned to their nearest controllers while considering latency of switches and load of controllers. Further, the algorithm relocates some of the switches from the controller to which they are currently assigned so as to reduce the latency between switches and controllers. The proposed algorithm is evaluated on widely used Chinanet and Interoute networks extracted from internet topology zoo. The results show that the proposed algorithm outperforms the existing controller placement solutions for software defined wide area networks in terms of load imbalance and inter controller latency without affecting the average and standard deviation in switch to controller latency.

1. Introduction

Software Defined Networking (SDN) achieves programmability of the control plane by separating it from the data plane [1]. The intelligent control plane is moved to an outside entity known as controller, however, the data plane responsible for forwarding stays within the network element. Typically, multiple controllers are physically distributed across the network to satisfy various performance metrics such as latency perceived by switches, fault tolerance, and controller response time. In a multi-controller architecture, each switch is assigned to at least one controller and each controller provides service to a set of switches [2–4]. The number and placement of controllers directly affects the aforementioned performance metrics. The controller placement problem (CPP) takes the network topology as input and typically generates the number of controllers needed for the network and their placement and switches assigned to each of them so as to minimize or maximize some performance metric. Heller et al. started the investigation of CPP and studied the effect of placement on the latency [5].

In long-propagation-delay software defined wide area networks (SD-WANs), the placement of controllers affects fundamental limits on availability and convergence time. When deploying SDN in real networks, large networks such as wide area network (WAN) are always partitioned into several smaller ones due to numerous reasons: privacy, scalability, incremental deployment, security and so on [6,7]. The central idea is to partition the WAN into multiple domains and assign a controller to each domain as depicted in Fig. 1. A domain can be a sub-network in a Data Center (DC), an enterprise network or an Autonomous System (AS). Throughout the paper, we use the terms zone, domain, subnetwork, and partition interchangeably, but they convey the same meaning.

Various approaches for effective deployment of controllers in WAN are explored in the literature. However, none of the strategies considered load imbalance among controllers. Network partitioning approaches that are oblivious to load produce partitions that are highly imbalance in nature. Consequently, controllers deployed in large partitions are being overloaded, hence, they are more prone to failures. Contrarily, controllers deployed in smaller partitions are lightly loaded, hence, they are underutilized.

Motivated from this, we propose a strategy for controller placement to minimize the maximum load imbalance between controllers while enforcing that the number of switches assigned to each controller is greater than a minimum value. Henceforth, it is referred as load balanced controller placement (LBCP) problem. We also propose a scalable algorithm for LBCP to compute near optimal solutions on large scale networks. Hereafter, it is referred as scalable controller placement with balancing constraints (SCPBC). The algorithm uses

* Corresponding author.
E-mail addresses: bprao@nitw.ac.in (B.R. Killi), svrao@iitg.ac.in (S.V. Rao).

https://doi.org/10.1016/j.comcom.2020.02.053
Received 9 January 2020; Accepted 18 February 2020
Available online 24 February 2020
0140-3664/© 2020 Elsevier B.V. All rights reserved.

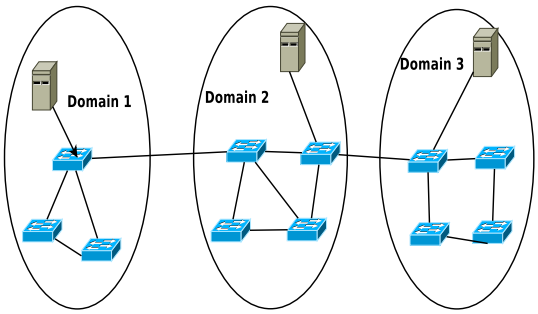


Fig. 1. Partitioning WAN into multiple domains.

poly-stable matching to distribute a fraction of switches equally among controllers so as to reduce the load imbalance. The remaining switches are assigned to their nearest controllers while considering latency of switches and load of controllers. Further, the algorithm relocates some of the switches from the controller to which they are currently assigned so as to reduce the latency between switches and controllers. The proposed SCPBC algorithm is evaluated on widely used networks such as Internet 2 OS3E, Chinanet and Interoute networks. The results show that the proposed algorithm outperforms the existing controller placement solutions for software defined wide area networks in terms of load imbalance and inter controller latency (ICL) without affecting the average switch to controller latency (SCL) and deviation in SCL.

The rest of the paper is organized as follows. Section 2 introduces related work on controller placement in SDN and formulate SCPBC in Section 3. The results obtained by evaluating the proposed algorithms on real word network topologies are presented in Section 4. Finally, we conclude the paper in the last section.

2. Related work

In this section, we discuss key contributions of the controller placement problem in SDN. The works on controller placement in SDN can be mainly classified into static and dynamic approaches. Further, a large section of works on CP in SDN focus on resiliency of network elements, hence, we discuss these approaches in the following subsections.

2.1. Controller placement in SDN

Sallahi et al. investigated the planning problem in SDN while taking diverse capabilities of controllers and links into account so as to minimize the cost of installing and linking controllers and connecting data plane and control plane elements [8]. The same authors investigated the expansion problem in SDN that takes the existing topology and list of new controllers and links as input and reorganizes the network so as to minimize the update cost of the existing network [9]. In [10,11], the authors proposed analytical models to capture the effect of control traffic on the reaction time perceived by the switches under two different data ownership scenarios. They also proposed a mathematical model and an evolutionary algorithm to minimize the reaction time perceived by the switches. The authors of [12] considered two conflicting metrics such as SCL and ICL and devised two different models to minimize one metric while constraining other metric. They used a bargaining game to analyze the trade-off between these two metrics.

2.2. Dynamic controller placement in SDN

Dynamic approaches for provisioning controllers and migrating switches according to the variation in switch traffic demands are explored in [13–19]. A protocol for migrating switches while enforcing serializability and algorithms for dynamic provisioning of controllers are proposed in [13,14]. They observed that enforcing serializability impacts the performance and analyzed their trade-off. The authors of [15] proposed a framework for provisioning controllers dynamically by monitoring the variation in switch traffic demands and reassigning some of the switches from existing controllers. The work in [16] formulated the dynamic provisioning of controllers as a cooperative non-zero sum game while taking controller utilization and delay into account. The authors of [17,18] utilized a maximum latency constraint to partition a WAN and proposed two location searching heuristics for placing controller modules in each partition. Further, they used regression technique to predict the traffic generated by switches at a time t to dynamically configure the number of controller instances within a controller module. In [19], the problem of periodically computing the assignment of switches to controllers based on the traffic variations to minimize response time and overhead is formulated as a stable matching. But they have not considered load imbalance among controllers. Different form the aforementioned work, we minimize the load imbalance among controllers while enforcing that the number of switches assigned to each controller is greater than a minimum value.

2.3. Resilient controller placement in SDN

Taking reliability of network elements into account, different solution approaches for placement of controllers are proposed in [20–38]. The works in [20–26] investigated the influence of switch and link failures and proposed models and heuristics for reducing the disconnections between control and data plane elements. In [27,28], several conflicting metrics with different failure scenarios such as controller, switch and link failure are considered and proposed a graphical framework and heuristic to generate pareto optimal solutions. Killi et al. observed a significant increase in the SCL along with disconnections as a consequence of controller failures [29,30]. They used multiple levels of backup controllers and resilient placement schemes to circumvent the after effects of controller failures. The authors of [31] investigated the effect of limiting SCL, ICL, and load imbalance on the number of controllers needed and formulated it as a linear program. A scalable algorithm, based on the idea of maximum clique of a graph, to reduce the number of controllers needed to provide a specific level of resilience is proposed in [32,33]. The works in [34,35] proposed different traffic distribution approaches to reduce the flow setup time under long term and short time controller failures. Different approaches to reduce the backup capacity needed to provide controller resilience are proposed in [37,38]. Lourenco et al. explored an approach for installing controllers in transport network under random and correlated link and controller failures [36]. They observed that using redundant controllers can provide resilience against random failures and effective placement of controllers can provide resilience against correlated failures. Further, they proposed an heuristic for reassigning disconnected switches after failures.

2.4. Controller placement in WAN

Various approaches for effective deployment of controllers in WAN are explored in the literature [39–44]. In [39], the problem of partitioning a WAN into smaller units known as zones is devised as a NP-hard optimization model that computes the optimal zone size while minimizing the overhead within and across zones. The authors of [40] utilized the spectral clustering to reduce the similarity across domains and improve the similarity within a domain. Liao et al. computed the minimum number of controllers needed for a given WAN using a density based clustering approach [41]. They have defined the density of a switch s_i as the number of switches whose distance to s_i is less than a threshold. Switches with higher density are selected as cluster heads and the remaining switches are assigned to the nearest high density switch cluster. In [42], the WAN partitioning problem is devised as a cooperative game to reduce the maximum switch to controller

latency. In this paper, this approach is referred to as cooperative game based controller placement (CGCP) for brevity. The authors of [43,44] leveraged a variation of k-means algorithm that progressively compute the subnetworks while reducing the latency of switches. In this paper, this approach is referred to as progressive k-means based controller placement (PKCP) for brevity. None of the aforementioned strategies considered load imbalance among controllers.

3. Problem formulation

We assume that controllers and switches can be co-located in the network, therefore, the potential locations L for installing controllers is same as the location of switches S, i.e., $L = S$. Let Q be the number of control domains in the network. We assume that each domain is controlled by a single controller. Therefore, Q denotes the number of controllers to be installed. The latency between switch s_i and controller c_k is represented by $\delta(s_i, c_k)$. Let the fraction $\alpha \in [0, 1]$ be the balance parameter that controls the minimum number of switches must be assigned to each controller.

3.1. Decision variables

We utilize a placement variable $Y_c, \forall c \in L$ and a mapping variable $A_{s,c}, \forall s \in S, \forall c \in L$ that are defined as follows:

$$Y_c = \begin{cases} 1, & \text{if a controller is placed at location } c \in L \\ 0, & \text{otherwise} \end{cases}$$

$$A_{s,c} = \begin{cases} 1, & \text{if the switch } s \in S \text{ is mapped to} \\ & \text{the controller placed at location } c \in L \\ 0, & \text{otherwise} \end{cases}$$

3.2. Optimization formulation

The goal of LBCP is to minimize the maximum load imbalance between controllers while enforcing the placement and balancing constraints. It is formulated as follows:

$$min \left\{ \max_{c \in L} \sum_{s \in S} A_{s,c} - \min_{c \in L} \sum_{s \in S} A_{s,c} \right\} \quad (1)$$

$$\sum_{c \in L} Y_c = Q \quad (2)$$

$$\sum_{c \in L} A_{s,c} = 1 \ \ \forall s \in S \quad (3)$$

$$A_{s,c} \leq Y_c \ \ \forall s \in S, \forall c \in L \quad (4)$$

$$\sum_{s \in S} A_{s,c} \geq \left\lfloor \alpha * \frac{|S|}{|C|} \right\rfloor \ \ \forall c \in L \quad (5)$$

$$A_{s,c} \in \{0, 1\} \ \ \forall s \in S, \forall c \in L \quad (6)$$

$$Y_c \in \{0, 1\} \ \ \forall c \in L \quad (7)$$

The objective function in (1) minimizes the maximum load imbalance among controllers. That is, it minimizes the difference between the number of switches assigned to the controller with maximum load and the controller with minimum load. (2), (3) and (4) are placement constraints, (5) is the load balancing constraint and (6) and (7) are domain constraints. (2) restricts the number of controllers deployed in the network to Q. (3) enforces unique switch to controller assignment, that is, each switch should be assigned to a unique controller. (4) enforces assignment of switch to a valid controller. (5) ensures that the number of switches assigned to a controller is greater than or equal to $\alpha * \frac{|S|}{|C|}$. (6) and (7) enforce that the variables $A_{s,c}$ and y_c take binary values (see Table 1).

However, the sub terms (max and min terms) in (1) are non linear. If we know the number of switches assigned to each controller, then the max and min terms in (1) can be linearized by introducing additional variables and constraints. Since the number of switches assigned to each controller is computed at runtime, the max and min terms in (1) remains non linear. Hence, the above optimization formulation is non-linear.

Table 1
Notations used in LBCP and SCPBC.

Symbol	Description
$G(S, E)$	Physical network
S	Set of switches in the network
E	Set of physical links
Q	Number of controllers to be installed.
L	Potential locations for installing controllers.
α	Balancing parameter $\in [0, 1]$: It decides the minimum number of switches need to be assigned to a controller
Π_k	Preference list of controller c_k
β_k	Preference list index of controller c_k
λ_k	Number of switches yet to be assigned to c_k to satisfy balancing constraint.
Δ_i	Preference list of switch s_i
L_0	Minimum number of switches need to be assigned to a controller
θ	Sum total of $\lambda_k, \forall c_k$
t	Number of switch relocations.
$\delta(s_i, c_k)$	Switch s_i to controller c_k latency.
Y_c	= 1 , if a controller is installed at c = 0, otherwise.
$A_{s,c}$	= 1, if switch s is mapped to controller at c. = 0, otherwise.

3.3. Scalable controller placement with balancing constraints

In this section, we present a scalable algorithm for the LBCP using poly-stable matching and switch relocations. It is refereed to as Scalable Controller Placement with Balancing Constraints (SCPBC). We also analyze the running time of the SCPBC algorithm.

Algorithm 1 describes the pseudo code of SCPBC algorithm. It takes as input set of switches S, a balance parameter $\alpha \in [0, 1]$, and two boolean parameters $useLSA$ and $useSR$ and produces as output controller locations C and switch to controller assignment A. Step 1 of the algorithm initializes the controller locations using a deterministic k-means initialization technique proposed in [45]. Then, step 2 computes the minimum number of switches need to be assigned to each controller. Typically, the ideal load of a controller is the ratio between the number of switches and the number of controllers in the network. However, assigning $\lfloor \alpha * (\frac{|S|}{|C_0|}) \rfloor$ switches to each controller enforces the balancing constraints. The preference list Π_k of a controller c_k is computed in step 4 by sorting all the switches $s_i \in S$ based on the latency from the controller. Step 5 initializes the controller preference list index β_k such that it points to the first element in the list. Step 6 initializes the number of switches yet to be assigned to a controller with L_0. The preference list Δ_i of a switch s_i is computed in step 9 by sorting all the controllers $c_k \in C$ based on the latency from the switch. In step 12, the poly stable matching algorithm is invoked to compute the switch to controller assignment of θ switches. The switch to controller assignment of the remaining $|S| - \theta$ switches is computed in steps 13–22. For each switch s_i that is not assigned to any controller, a nearest controller of the switch is computed in step 16 if the input boolean parameter with LSA is zero, otherwise a load sensitive nearest controller of the switch is computed in step 18. The switch to controller assignment is updated in step 20. Finally, steps 23–36 relocates some switches depending on the value of useSR boolean parameter to reduce the maximum switch to controller latency of the solution without violating the balancing constraints. The number of relocated switches t and updated controller locations C^* are initialized to zero and empty respectively. Step 27–30 of the algorithm updates the controllers within each subnetwork. Note that the controller along with all the switches assigned to it forms a subnetwork or cluster. Then,

Algorithm 1: Scalable controller placement with balancing constraints

Input: S, α, $useLSA$, $useISR$
Output: Controller locations

1 C_0 = Initial controller locations (S)
2 $L_0 = \lfloor \alpha * (\frac{|S|}{|C_0|}) \rfloor$
3 **for** k =1: *to* $|C_0|$ **do**
4 $\quad \Pi_k$ = Sort switches based on $\delta(s_i, c_k), \forall s_i \in S$
5 $\quad \beta_k = 1$
6 $\quad \lambda_k = L_0$
7 **end**
8 **for** i =1: *to* $|S|$ **do**
9 $\quad \Delta_i$ = Sort controllers based on $\delta(s_i, c_k), \forall c_k \in C_0$
10 **end**
11 $\Theta = \sum_{k=1}^{|C_0|} \lambda_k$
12 $[A]$ = PolyStableMatching (S, C_0, Π, Δ, β, λ, Θ,L_0)
13 **for** i =1 *to* $|S|$ **do**
14 $\quad$ **if** s_i *is free* **then**
15 $\quad\quad$ **if** *useLSA* ==0 **then**
16 $\quad\quad\quad$ nc = NearestController (s_i, C, D)
17 $\quad\quad$ **else**
18 $\quad\quad\quad$ nc = nearestLSController (s_i, C, D, A)
19 $\quad\quad$ **end**
20 $\quad\quad A_{nc} = A_{nc} \cup \{s_i\}$
21 $\quad$ **end**
22 **end**
23 **if** *useISR* ==1 **then**
24 $\quad$ **repeat**
25 $\quad\quad t = 0$
26 $\quad\quad C^* = \phi$
27 $\quad\quad$ **for** $k = 1$ *to* $|C|$ **do**
28 $\quad\quad\quad m = \arg \min_{i:i \in A_k} \sum_{l:l \in A_k} d(s_i, s_l)$
29 $\quad\quad\quad C^* = C^* \cup \{m\}$
30 $\quad\quad$ **end**
31 $\quad\quad C = C^*$
32 $\quad\quad [A, \gamma, t]$ = IndividualSwitchRelocation (S, C, A, D, γ, t)
33 $\quad\quad G$ = ConstructAssignmentGraph(C, M, γ)
34 $\quad\quad [G, A, t]$ = GroupSwitchRelocation (G, A, t)
35 $\quad$ **until** $t = 0$;
36 **end**

Algorithm 2: Nearest LS Controller

Input: switch index i, S, C, D, A
Output: nc

1 minLD = ∞
2 **for** $k = 1$ *to* $|C|$ **do**
3 $\quad$ LDvalue = $\delta(s_i, c_k) * |A_k|$
4 $\quad$ **if** *LDvalue* < *minFD* **then**
5 $\quad\quad$ minLD = LDvalue
6 $\quad\quad nc = k$
7 $\quad$ **end**
8 **end**

individual switch relocation and group switch relocation algorithms are invoked in step 32 and step 34 respectively. The assignment graph generated in step 33 is used by the group switch relocation algorithm in step 34.

The pseudo code of computing nearest load sensitive controller of a switch is presented in Algorithm 2. It takes a switch, set of controllers, delay matrix, and switch to controller assignment as input and computes the nearest controller of the switch while considering both delay and load (number of switches assigned to a controller). the "LDvalue" of switch i and controller c_k is computed as a product of the latency between s_i and c_k and load of the controller c_k in step 3. The controller with the minimum LDvalue is computed between steps 1–8. Note that, the algorithm for computing nearest controller of a switch is similar to Algorithm 2, but it does not consider load of controllers in step 3.

Algorithm 3: Poly_Stable_Matching

Input: S, C, Π, Δ, β, λ, Θ, L_0
Output: clusters: A

1 **for** k =1 *to* $|C|$ **do**
2 $\quad A_k = \phi$
3 **end**
4 Mark all switches $s_i \in S$ as free
5 **while** $\Theta > 0$ **do**
6 $\quad$ **for** k =1 *to* $|C|$ **do**
7 $\quad\quad$ **if** $\lambda_k > 0$ **then**
8 $\quad\quad\quad \lambda_k^* = \lambda_k$
9 $\quad\quad\quad$ **for** $q = \beta_k$ *to* $(\beta_k + \lambda_k - 1)$ **do**
10 $\quad\quad\quad\quad$ **if** $\Pi_k(q)$ *is free* **then**
11 $\quad\quad\quad\quad\quad A_k \leftarrow A_k \cup \{\Pi_k(q)\}$
12 $\quad\quad\quad\quad\quad$ Mark $\Pi_k(q)$ as engaged to c_k
13 $\quad\quad\quad\quad\quad \lambda_k = \lambda_k - 1$
14 $\quad\quad\quad\quad$ **else if** $\Pi_k(q)$ *is engaged to controller* $c_{k'}$ **then**
15 $\quad\quad\quad\quad\quad$ **if** $\Pi_k(q)$ *prefers* c_k *over* $c_{k'}$ **then**
16 $\quad\quad\quad\quad\quad\quad A_k \leftarrow A_k \cup \{\Pi_k(q)\}$
17 $\quad\quad\quad\quad\quad\quad A_{k'} \leftarrow A_{k'} \setminus \{\Pi_k(q)\}$
18 $\quad\quad\quad\quad\quad\quad$ Mark $\Pi_k(q)$ as engaged to c_k
19 $\quad\quad\quad\quad\quad\quad \lambda_k = \lambda_k - 1$
20 $\quad\quad\quad\quad\quad\quad \lambda_k' = \lambda_k' + 1$
21 $\quad\quad\quad\quad\quad$ **end**
22 $\quad\quad\quad\quad$ **end**
23 $\quad\quad\quad$ **end**
24 $\quad\quad\quad \beta_k = \beta_k + \lambda_k^*$
25 $\quad\quad$ **end**
26 $\quad$ **end**
27 $\quad \Theta = \sum_{k=1}^{|C|} \lambda_k$
28 **end**

Algorithm 3 describes the pseudo code for the poly stable matching. It takes as input set of switches S, set of controllers C, preference lists Δ and Π, controller preference list indexes β, vector comprising of the minimum number of switches yet to be assigned to each controller λ and the minimum number of switches need to be assigned to each controller L_0. For every controller, steps 1–3 of the algorithm initializes the set of switches assigned to empty. Then, all the switches in the network are marked as free. StepS 5–28 of the algorithm computes the switch to controller assignment of θ switches. In StepS 9–23, each controller c_k proposes to next β_k switches in the preference list if the number of switches yet to be assigned to c_k is greater than zero, i.e., $\lambda_k > 0$. When a free switch gets a proposal from a controller, it immediately accepts the proposal, engages to that controller, and updates the switch to controller assignment. If the switch is already engaged to some controller and gets a proposal from some other controller, then the switch to controller assignment is updated based on the switch preference list. The number of switches yet to be assigned to a controller is updated in stepS 13, 19, and 20 depending on whether the proposal is accepted by a switch or not. Then, the controller preference list index is updated in step 24. Finally, the value of theta is computed

Algorithm 4: Individual_Switch_Relocation

```
Input: S, C, A, D, t
Output: A, M, γ, t
1  for i = 1 to |S| do
2      γ(s_i) = ϕ
3      for q = 1 to |C| do
4          h = M(s_i)
5          if δ(s_i, c_q) < δ(s_i, c_h) then
6              if |A_h| > λ_h then
7                  A_q ← A_q ∪ {s_i}
8                  A_h ← A_h \ {s_i}
9                  t = t + 1
10             else
11                 γ(s_i) = γ(s_i) ∪ {c_q}
12             end
13         end
14     end
15 end
```

Algorithm 5: Construct_Assignment_Graph

```
Input: S, C, γ
Output: G
1  for k = 1 to |C| do
2      for q = 1 to |C| do
3          G(k, q) = 0
4      end
5  end
6  for i = 1 to |S| do
7      h = M(s_i)
8      for q ∈ γ(s_i) do
9          G(h, q) = G(h, q) + 1
10     end
11 end
```

using updated λ_k values in step 27. Steps 6–27 are repeated until the value of θ is zero.

The pseudo code of individual switch relocation (ISR) is presented in Algorithm 4. For a switch s_i, $\gamma(s_i)$ denotes the set of controllers that are closer than the controller to which s_i is currently assigned. A switch s_i is relocated if there exists a controller c_q that is closer than the controller c_h to which it is currently assigned and c_h can afford to let go a switch without violating the balancing constraints. Consequently, steps 7–9 update the switch to controller assignment and the number of reassigned switches. On the other hand, if there exists a controller c_q that is closer than the controller c_h to which it is currently assigned but c_h cannot afford to let go a switch without violating the constraints, then step 11 updates $\gamma(s_i)$.

Algorithm 5 describes the pseudo code for constructing directed potential assignment graph. It takes as input set of controllers C, set of switches S, and $\gamma(s_i)$ for each switch. For every controller, a node is introduced in the graph. Steps 1–5 initialize the adjacency matrix with zeros. For every switch s_i, steps 6–11 introduce a directed edge of unit weight between the controller to which s_i is currently assigned and every other controller in the set $\gamma(s_i)$. The cumulative weight of an edge between two nodes is the sum total of weights of all edges introduced between those nodes.

The pseudo code of group switch relocation (GSR) is presented in Algorithm 6. It takes the potential assignment graph as input and returns updated switch to controller assignment and updated number of switch reassignments. Step 1 computes the strongly connected components (SCCs) of the potential assignment graph G. Then, the algorithm applies depth first search (DFS) traversal on each component ω and detect

Algorithm 6: Group_Switch_Relocation

```
Input: G, A, t
Output: A, t
1  F =Compute Strongly_Connected_Components(G)
2  for ω ∈ F do
3      while there exists back edge in DFS (ω) do
4          μ = Cycle corresponding to this back edge
5          t = t+ minimum edge weight(μ)
6          μ_t = μ with all edge weights set to t
7          Reassign switches according to μ_t
8          ω ← ω \ μ_t
9      end
10 end
```

Table 2
Characteristics of input networks.

Network/ Parameter	OS3E	Chinanet	Interoute
Nodes	34	42	110
Edges	42	66	158
Min degree	1	1	1
Max degree	5	20	7
Avg degree	2.47	3.14	2.87
Diameter	25.4 ms	35.75 ms	20.47 ms

cycles corresponding to each back edge in DFS of ω. The cycles from the components are removed after performing switch reassignments.

Theorem 1. *The running time of SCPBC algorithm is* $O(|C||S|log|S|)$.

Proof. Step 4 of the algorithm sorts switches which takes $O(|S|log|S|)$ time. Consequently, steps 3–7 take $O(|C||S|log|S|)$ time. Step 9 of the algorithm sorts controllers which takes $O(|C|log|C|)$ time. Consequently, steps 8–10 take $O(|S||C|log|C|)$ time. Since $C \subseteq S$, step 3–10 take $O(|C||S|log|S|)$ time. Each controller in Algorithm 3 proposes each of the θ switches atmost once, hence, the running time of step 12 is $O(|C|\theta)$. Each of the remaining $|S|-\theta$ switches is assigned to their nearest controllers while considering load and/or latency, whose running time is $O((|S| - \theta).|C|)$. Updating controller locations in steps 27–30 take $O(|S||C|)$ time. Since Algorithm 4 checks each switch against each controller for a possible relocation, the running time of individual switch relocation is $O(|S||C|)$. Algorithm 5 introduces an edge between each switch and each controller in $\gamma(s_i)$, hence, the running time of constructing potential assignment graph is $O(|S||C|)$. Since the nodes in an assignment graph corresponds to controllers and Algorithm 6 uses DFS to remove cycles and reassign switches, the running time of Algorithm 6 is $O(|C||S| + t_1|C|^2)$ where t_1 is the number of SCCs. Note that $O(t_1|C|^2)$ time for DFS and $O(|C||S|)$ time for reassignment of switches while removing cycles. Hence, the total running time of SCPBC algorithm is $O(|C||S|log|S|)$. □

4. Evaluation results

The proposed SCPBC algorithm is evaluated on three widely used real world ISP networks extracted from Internet 2 OS3E [46] and Internet topology zoo [47] and their characteristics are described in Table 2. The first topology is a medium scale network with 34 nodes and 42 edges, the second topology is a medium scale network with 42 nodes and 66 edges and the third topology is a large scale network with 110 nodes and 158 edges. The link weights and shortest paths between every pair of nodes is computed by extracting the position information of nodes from the graphml files. Medium and large scale topologies are considered to investigate the scalability and effectiveness of our proposed approach in WAN.

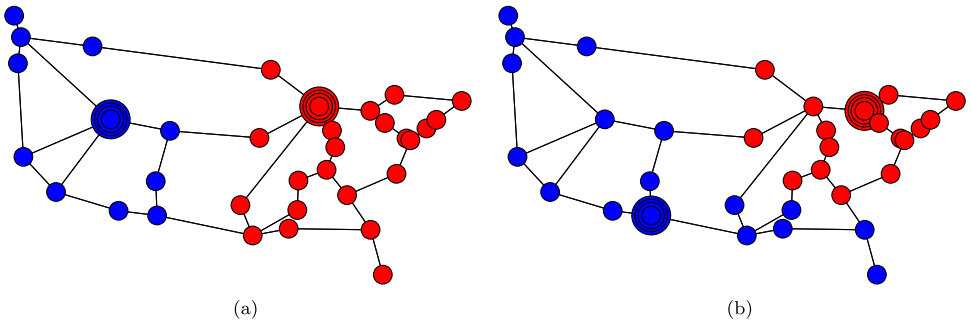


Fig. 2. Dividing Internet 2 OS3E topology into two partitions using SCPBC algorithm. (a) $\alpha = 0.5$. (b) $\alpha = 1$. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

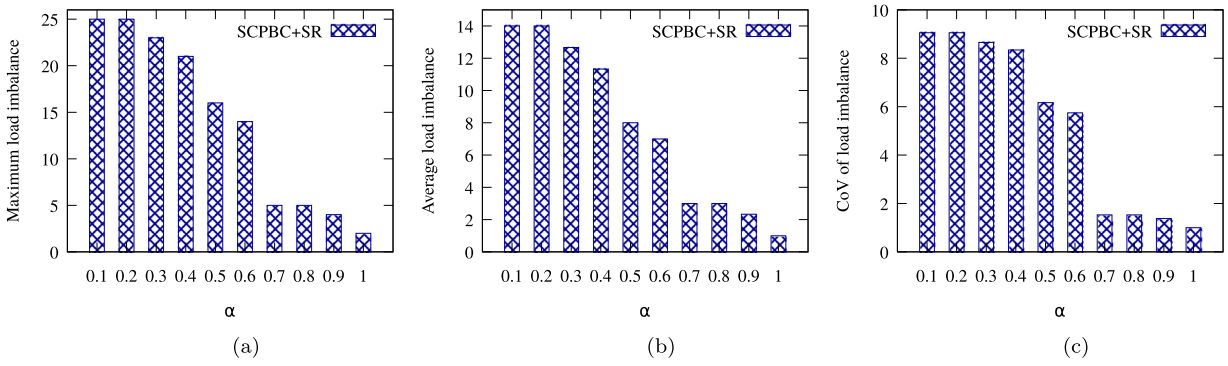


Fig. 3. Impact of α on load imbalance of chinanet topology (a) Maximum imbalance. (b) Average imbalance. (c) Standard deviation of imbalance.

The Internet 2 OS3E topology is divided into two subnetworks using the SCPBC algorithm with the input boolean parameters set to one. Fig. 2(a) and Fig. 2(b) illustrates the partitioning of the network and placement of controller in each partition when the balance parameter α is set to 0.5 and 1 respectively. Switches within a partition are shown with same color and the controller is highlighted with encircled node of same color. We can observe that the partitions in Fig. 2(a) are balanced up to some extent but not completely balanced. Further, the partitions in Fig. 2(b) are balanced, that is almost equal number of switches are there in each partition.

Figs. 3 and 4 illustrates the influence of balance parameter α on load imbalance when evaluated on chinanet and interoute topologies receptively. The balance parameter α is varied from 0.1 to one. Four and six subnetworks are considered while evaluating chinanet and interoute networks respectively. SCPBC algorithm equally distributes θ switches among k controllers using poly stable matching and remaining $|S| - \theta$ switches are assigned to their nearest controllers.

from step 11 of Algorithm 3

$$\theta = \sum_{k=1}^{|C|} \lambda_k \tag{8}$$

from step 6 of Algorithm 3

$$\theta = \sum_{k=1}^{|C|} L_0 \tag{9}$$

$$= |C| * L_0 \tag{10}$$

from step 2 of Algorithm 3

$$\theta = |C| * \alpha * \frac{|S|}{|C|} \tag{11}$$

$$= \alpha * |S| \tag{12}$$

While fixing the number of controllers and switches in the network, the value of θ directly depends on the value of α. That is, the value of θ increases with higher values of α. Hence, more number of switches are equally distributed among controllers for higher values of α, thereby, potentially reducing the imbalance among controllers. Therefore, we can observe from Figs. 3(a) and 3(b) that the maximum and average load imbalance decreases with the higher values of α. The deviation in load imbalance also decreases with the higher values of α which can be observed from Fig. 3(c) Further, we can observe that there is a significant reduction in load imbalance among controllers on chinanet and interoute networks when the value of $alpha$ is more than 0.6 and 0.5 respectively.

Table 3
Variations of SCPBC.

useLSA	useSR	SCPBC variation
False	False	SCPBC
True	False	SCPBC + LSA
False	True	SCPBC + SR
True	True	SCPBC + LSA + SR

We evaluated different variations of SCPBC algorithm presented in Table 3 based on the values of boolean input parameters useLSA and useSR. We have also evaluated the performance of CGCP from [42] and PKCP from [43] as baseline algorithms for comparison with the SCPBC algorithm. These baseline algorithms divide the network using a variation of clustering algorithm as discussed in Section 1.

The load imbalance of CGCP, PKCP, and SCPBC algorithms when evaluated on chinanet and interoute networks is demonstrated in Fig. 5 and Fig. 6 respectively. The value of α is set to 0.7 while evaluating SCPBC algorithm. Four and five subnetworks are considered on chinanet topology and six and seven subnetworks are considered on interoute topology. We can observe from Figs. 5(a) and 6(a) that the maximum load imbalance of different variations of SCPBC is much lower than CGCP and PKCP because SCPBC with distributes fraction of the switches while considering load of controllers whereas CGCP

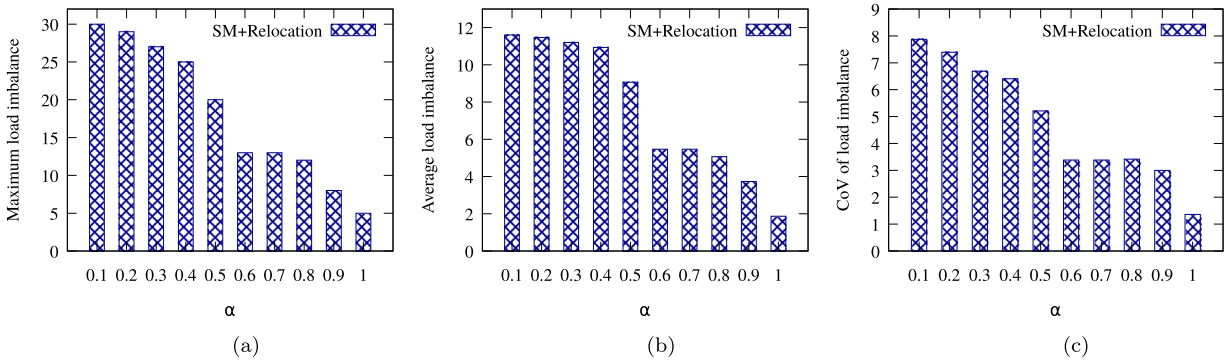


Fig. 4. Impact of α on load imbalance of interoute topology (a) Maximum load imbalance. (b) Average load imbalance. (c) Standard deviation of load imbalance.

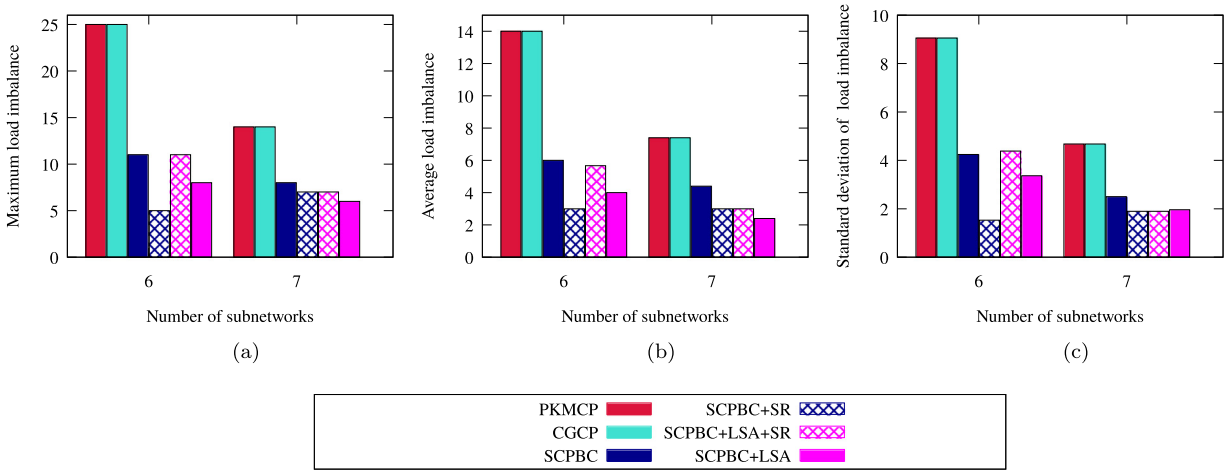


Fig. 5. Load imbalance comparison of CGCP, PKCP, SCPBC algorithms on chinanet topology. (a) Maximum imbalance. (b) Average imbalance (c) Standard deviation of imbalance.

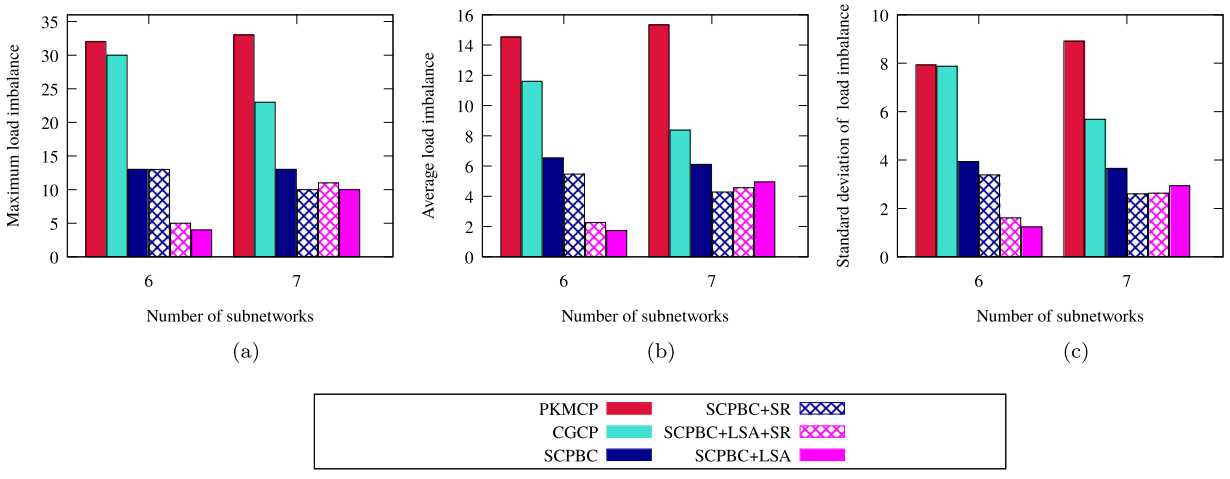


Fig. 6. Load imbalance comparison of CGCP, PKCP, SCPBC algorithms on interoute topology. (a) Maximum imbalance. (b) Average imbalance (c) Standard deviation of imbalance.

and PKCP create subnetworks to reduce the latency of switches without considering the load of controllers. Since the maximum load imbalance of a network does not capture the imbalance across all controllers, we have presented the average load imbalance and standard deviation of load imbalance in Fig. 5(b) and Fig. 6(b) and, Fig. 5(c) and Fig. 6(c) respectively. We can observe that the average load imbalance and standard deviation in imbalance of SCPBC is also lower than the CGCP and PKCP approaches. Further, we can also observe that the SCPBC variations with useLSA=1 performs better than the basic SCPBC with useLSA=0 and useSR=0. Because SCPBC with useLSA=1 assigns the remaining $|S|-\theta$ switches to their nearest controllers while considering delay and load whereas SCPBC with useLSA=0 and useSR=0 assigns the remaining $|S|-\theta$ switches to their nearest controllers without considering load of controllers. Since the SCPBC variation with useSR=1

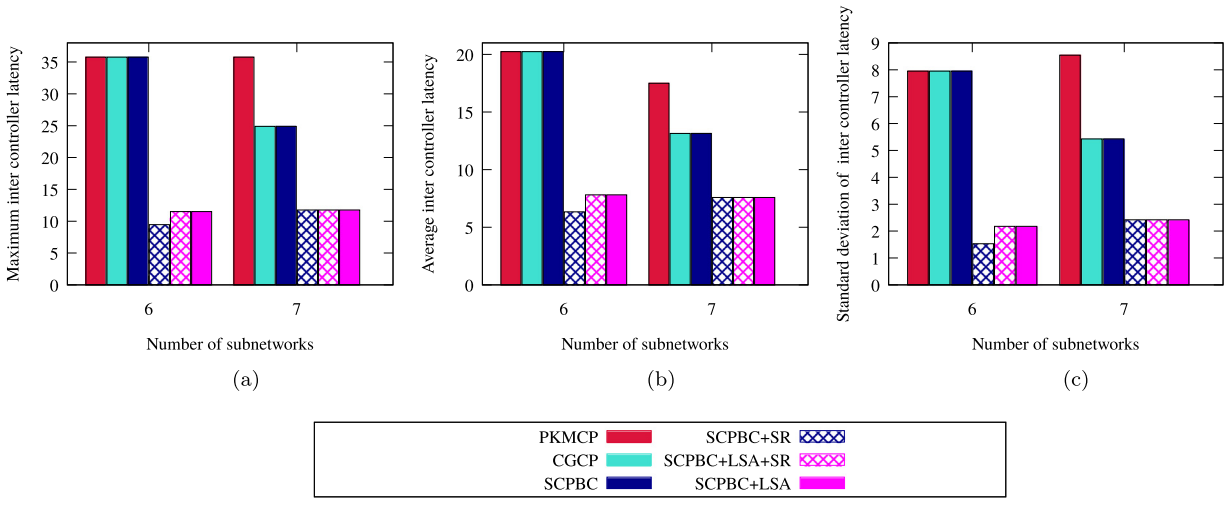


Fig. 7. Inter controller latency comparison of CGCP, PKCP, SCPBC algorithms on chinanet topology. (a) Maximum latency. (b) Average latency. (c) Standard deviation of latency.

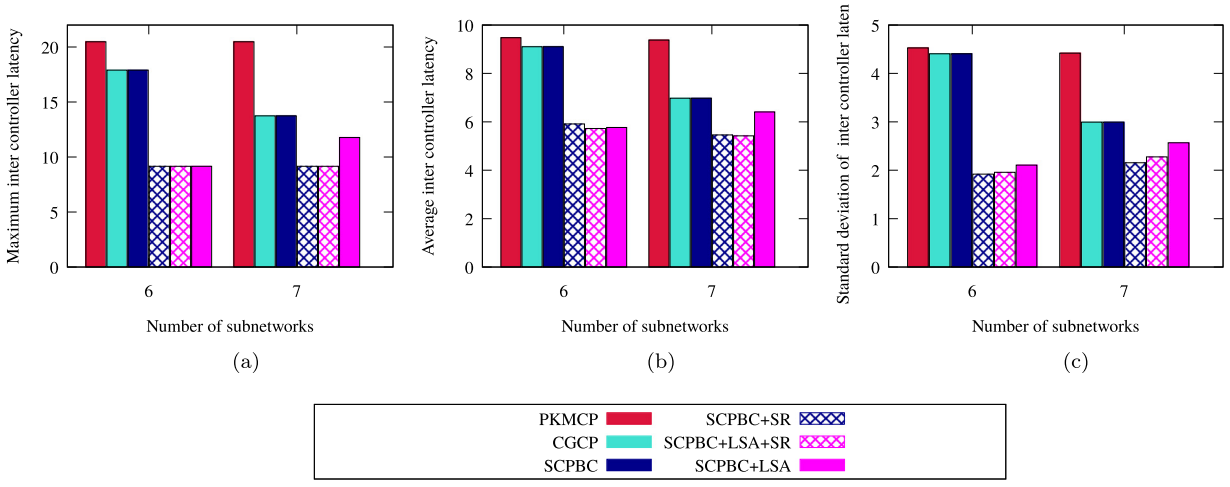


Fig. 8. Inter controller latency comparison of CGCP, PKCP, SCPBC algorithms on interoute topology. (a) Maximum latency. (b) Average latency. (c) Standard deviation of latency.

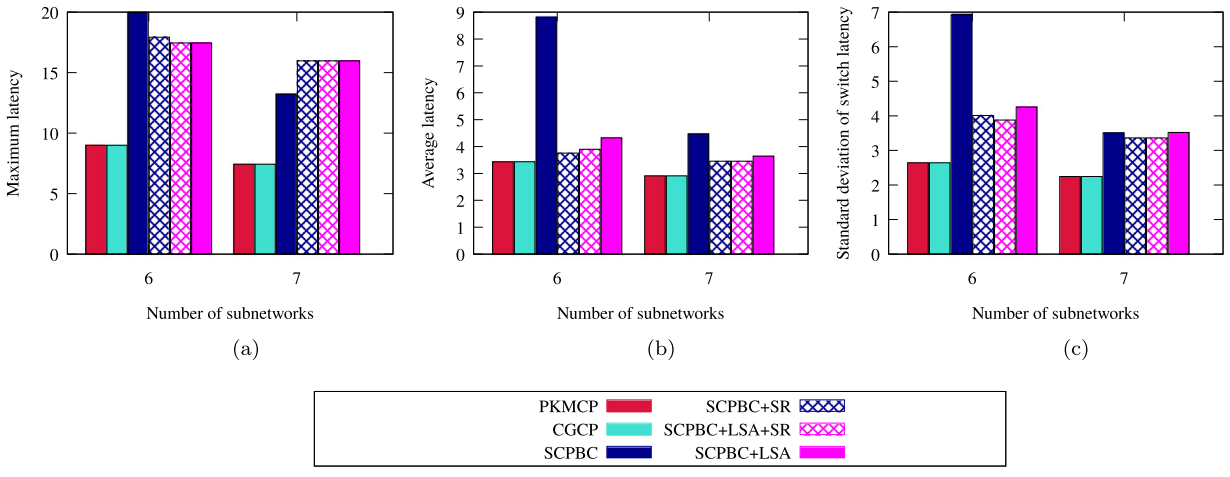


Fig. 9. Switch to controller latency comparison of CGCP, PKCP, SCPBC algorithms on chinanet topology (a) Maximum latency. (b) Average latency. (c) Standard deviation of latency.

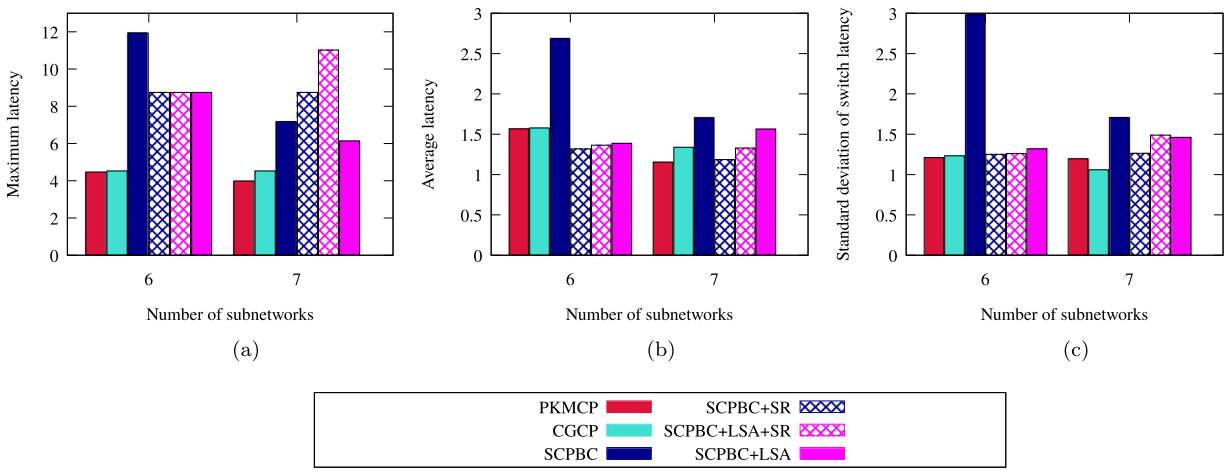


Fig. 10. Switch to controller latency comparison of CGCP, PKCP, SCPBC algorithms on interoute topology. (a) Maximum latency. (b) Average latency. (c) Standard deviation of latency.

performs individual and group switch relocations to further reduce the load imbalance, we can observe that its performance is also better than the SCPBC with useLSA=0 and useSR=0.

The latency between controllers directly influences the synchronization and convergence time in a WAN. Fig. 7 and Fig. 8 illustrates the inter controller latency of CGCP, PKCP, and SCPBC algorithms when evaluated on chinanet and interoute respectively. We can observe that the maximum, average, and deviation in inter controller latency of SCPBC is much lower than the CGCP and PKCP approaches. Further, we can also observe that the SCPBC variations with useLSA=1 outperforms the variations with useLSA=0.

The switch to controller latency of CGCP, PKCP, and SCPBC algorithms when evaluated on chinanet and interoute networks is demonstrated in Fig. 9 and Fig. 10 respectively. We can observe from Figs. 9(a) and 10(a) that the maximum switch latency of CGCP and PKCP is lower than SCPBC because CGCP and PKCP focus on reducing maximum latency while creating subnetworks whereas SCPBC distributes the switches so as to reduce the load imbalance among controllers. Since the maximum switch latency does not take all switches into account, we have presented the average and standard deviation of switch latencies in Fig. 9(b) and Fig. 10(b) and, Fig. 9(c) and Fig. 10(c) respectively. We can observe that the average and standard deviation of switch latencies of SCPBC is nearly same as that of the CGCP and PKCP approaches. Hence, we argue that SCPBC does not affect the latencies perceived by the switches in the network.

5. Conclusion

We investigated the load balanced controller placement for a WAN and formulated it as a mathematical model while enforcing that the number of switches assigned to each controller is greater than a minimum value. Due to the non linear nature of the model, we proposed a scalable algorithm to solve the problem on large scale networks. It uses poly-stable matching to distribute a fraction of switches equally among controllers towards ensuring the balancing constraints. The remaining switches are assigned to their nearest controllers while considering latency of switches and load of controllers. Further, the algorithm relocates some of the switches from the controllers to which they are currently assigned so as to reduce the latency between switches and controllers. The algorithm along with state of the art baseline algorithms are evaluated on widely used chinanet and interoute networks from Internet topology zoo. Results demonstrated that the SCPBC algorithm outperforms the existing controller placement solutions for WAN in terms of load imbalance and ICL without affecting the average SCL and deviation in SCL.

Acknowledgments

The authors would like to thank the associate editor and the anonymous reviewers for many valuable suggestions and comments that greatly improved the paper.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] S. Sezer, S. Scott-Hayward, P.K. Chouhan, B. Fraser, D. Lake, J. Finnegan, N. Viljoen, M. Miller, N. Rao, Are we ready for SDN? Implementation challenges for software-defined networks, IEEE Commun. Mag. 51 (7) (2013) 36–43.

[2] A. Tootoonchian, Y. Ganjali, Hyperflow: A distributed control plane for openflow, in: Proc. Internet Network Management Conference on Research on Enterprise Networking, INM/WREN'10, 2010, p. 3.

[3] S. Hassas Yeganeh, Y. Ganjali, Kandoo: A framework for efficient and scalable offloading of control applications, in: Proceedings of the First Workshop on Hot Topics in Software Defined Networks, HotSDN '12, 2012, pp. 19–24.

[4] T. Koponen, M. Casado, N. Gude, J. Stribling, L. Poutievski, M. Zhu, R. Ramanathan, Y. Iwata, H. Inoue, T. Hama, S. Shenker, Onix: A distributed control platform for large-scale production networks, in: Proceedings of the 9th USENIX Conference on Operating Systems Design and Implementation, OSDI'10, 2010, pp. 351–364.

[5] B. Heller, R. Sherwood, N. McKeown, The controller placement problem, in: Proc. First Workshop on Hot Topics in Software Defined Networks, 2012, pp. 7–12.

[6] H. Xie, T. Tsou, H. Yin, D. Lopez, C. Kolias, Use cases for alto with software defined networks, 2012, IETF Internet-Draft.

[7] P. Lin, J. Bi, Y. Wang, East-west bridge for SDN network peering, in: Frontiers in Internet Technologies: Second CCF Internet Conference of China, ICoC, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, pp. 170–181.

[8] A. Sallahi, M. St-Hilaire, Optimal model for the controller placement problem in software defined networks, 19, 2015, pp. 30–33.

[9] A. Sallahi, M. St-Hilaire, Expansion model for the controller placement problem in software defined networks, IEEE Commun. Lett. 21 (2) (2017) 274–277.

[10] T. Zhang, A. Bianco, P. Giaccone, The role of inter-controller traffic in SDN controllers placement, in: Proceedings of the IEEE Conference on Network Function Virtualization and Software Defined Networks, NFV-SDN '16, IEEE, 2016, pp. 87–92, http://dx.doi.org/10.1109/NFV-SDN.2016.7919481.

[11] T. Zhang, P. Giaccone, A. Bianco, S.D. Domenico, The role of the inter-controller consensus in the placement of distributed SDN controllers, Comput. Commun. 113 (Suppl. C) (2017) 1–13.

[12] A. Ksentini, M. Bagaa, T. Taleb, I. Balasingham, On using bargaining game for Optimal Placement of SDN controllers,

[13] A. Dixit, F. Hao, S. Mukherjee, T. Lakshman, R. Kompella, Towards an elastic distributed SDN controller, SIGCOMM Comput. Commun. Rev. 43 (4) (2013) 7–12.

[14] A.A. Dixit, F. Hao, S. Mukherjee, T. Lakshman, R. Kompella, ElastiCon: An elastic distributed sdn controller, in: Proc. ACM/IEEE Symposium on Architectures for Networking and Communications Systems, 2014, pp. 17–28.
[15] M.F. Bari, A.R. Roy, S.R. Chowdhury, Q. Zhang, M.F. Zhani, R. Ahmed, R. Boutaba, Dynamic controller provisioning in software defined networks, in: Proc. International Conference on Network and Service Management, CNSM 2013, 2013, pp. 18–25.
[16] H.K. Rath, V. Revoori, S.M. Nadaf, A. Simha, Optimal controller placement in Software Defined Networks (SDN) using a non-zero-sum game, Proc. International Symposium on a World of Wireless, Mobile and Multimedia Networks, WoWMoM, 2014, pp. 1–6.
[17] M.T.I. ul Huque, G. Jourjon, V. Gramoli, Revisiting the controller placement problem, in: Proc. IEEE 40th Conference on Local Computer Networks, LCN, 2015, pp. 450–453.
[18] M.T.I. ul Huque, W. Si, G. Jourjon, V. Gramoli, Large-scale dynamic controller placement, IEEE Trans. Netw. Serv. Manag. 14 (1) (2017) 63–76.
[19] T. Wang, F. Liu, J. Guo, H. Xu, Dynamic SDN controller assignment in data center networks: Stable matching with transfers, in: IEEE International Conference on Computer Communications, INFOCOM, 2016, pp. 1–9.
[20] F.J. Ros, P.M. Ruiz, Five nines of southbound reliability in software-defined networks, in: Proc. Third Workshop on Hot Topics in Software Defined Networking, 2014, pp. 31–36.
[21] L.F. Muller, R.R. Oliveira, M.C. Luizelli, L.P. Gaspary, M.P. Barcellos, Survivor: an enhanced controller placement strategy for improving SDN survivability, in: Proc. IEEE GLOBECOM, 2014, pp. 1909–1915.
[22] Y. nan Hu, W. dong Wang, X. yang Gong, X. rong Que, S. duan Cheng, On the placement of controllers in software-defined networks, J. China Univ. Posts Telecommun. 19 (2) (2012) 92–171.
[23] Y. Zhang, N. Beheshti, M. Tatipamula, On resilience of split-architecture networks, in: Proc. IEEE GLOBECOM, 2011, pp. 1–6.
[24] M. Guo, P. Bhattacharya, Controller placement for improving resilience of software-defined networks, in: Proc. IEEE ICNDC, 2013, pp. 23–27.
[25] Y. Hu, W. Wang, X. Gong, X. Que, S. Cheng, On reliability-optimized controller placement for Software-Defined Networks, China Commun. 11 (2014) 38–54.
[26] F.J. Ros, P.M. Ruiz, On reliable controller placements in Software-Defined Networks, Comput. Commun. 77 (2016) 41–51.
[27] D. Hock, S. Gebert, M. Hartmann, T. Zinner, P. Tran-Gia, POCO-framework for Pareto-optimal resilient controller placement in SDN-based core networks, in: Proc. IEEE Network Operations and Management Symposium, NOMS, 2014, pp. 1–2.
[28] S. Lange, S. Gebert, T. Zinner, P. Tran-Gia, D. Hock, M. Jarschel, M. Hoffmann, Heuristic approaches to the controller placement problem in large scale SDN networks, IEEE Trans. Netw. Serv. Manag. 12 (2015) 4–17.
[29] B.P.R. Killi, S.V. Rao, Optimal model for failure foresight capacitated controller placement in software defined networks, 20 (6), 2016, pp. 1108–1111.
[30] B.P.R. Killi, S.V. Rao, Capacitated next controller placement in software defined networks, IEEE Trans. Netw. Serv. Manag. PP (99) (2017) 1.
[31] N. Perrot, T. Reynaud, Optimal placement of controllers in a resilient SDN architecture, in: Proceedings of the 12th International Conference on the Design of Reliable Communication Networks (DRCN), DRCN '16, IEEE, 2016, pp. 145–151, http://dx.doi.org/10.1109/DRCN.2016.7470849.
[32] M. Tanha, D. Sajjadi, J. Pan, Enduring node failures through resilient controller placement for software defined networks, in: Proceedings of the Global Communications Conference, GLOBECOM '16, IEEE, 2016, pp. 1–7, http://dx.doi.org/10.1109/GLOCOM.2016.7841786.
[33] M. Tanha, D. Sajjadi, R. Ruby, J. Pan, Capacity-aware and delay-guaranteed resilient controller placement for software-defined WANs, IEEE Trans. Netw. Serv. Manage. 15 (3) (2018) 991–1005.
[34] V. Sridharan, M. Gurusamy, T. Truong-Huu, On multiple controller mapping in software defined networks with resilience constraints, IEEE Commun. Lett. 21 (8) (2017) 1763–1766.
[35] V. Sridharan, M. Gurusamy, T. Truong-Huu, Multi-controller traffic engineering in software defined networks, in: Proceedings of the 42nd Conference on Local Computer Networks, LCN '17, IEEE, 2017, pp. 137–145, http://dx.doi.org/10.1109/LCN.2017.77.
[36] R.B. Lourenço, S.S. Savas, M. Tornatore, B. Mukherjee, Robust hierarchical control plane for Transport Software-Defined Networks, Opt. Switch. Netw. 30 (2018) 10–22.
[37] P.M. Mohan, T. Truong-Huu, M. Gurusamy, Primary-backup controller mapping for byzantine fault tolerance in software defined networks, in: Proceedings of the IEEE Global Communications Conference, in: GLOBECOM '17, IEEE, 2017, pp. 1–7, http://dx.doi.org/10.1109/GLOCOM.2017.8254755.
[38] B.P.R. Killi, S.V. Rao, Towards improving resilience of controller placement with minimum backup capacity in software defined networks, Comput. Netw. 149 (2019) 102–114.
[39] X. Li, P. Djukic, H. Zhang, Zoning for hierarchical network optimization in software defined networks, in: 2014 IEEE Network Operations and Management Symposium, NOMS, 2014, pp. 1–8.
[40] P. Xiao, W. Qu, H. Qi, Z. Li, Y. Xu, The SDN controller placement problem for WAN, in: Proc. IEEE/CIC International Conference on Communications in China, ICCC, 2014, pp. 220–224.
[41] J. Liao, H. Sun, J. Wang, Q. Qi, K. Li, T. Li, Density cluster based approach for controller placement problem in large-scale software defined networkings, Comput. Netw. 112 (2017) 24–35.
[42] B.P.R. Killi, E.A. Reddy, S.V. Rao, Cooperative game theory based network partitioning for controller placement in SDN, in: 2018 10th International Conference on Communication Systems Networks, COMSNETS, 2018, pp. 105–112.
[43] G. Wang, Y. Zhao, J. Huang, Q. Duan, J. Li, A K-means-based network partition algorithm for controller placement in software defined network, in: 2016 IEEE International Conference on Communications, ICC, 2016, pp. 1–6.
[44] G. Wang, Y. Zhao, J. Huang, Y. Wu, An effective approach to controller placement in software defined wide area networks, IEEE Trans. Netw. Serv. Manag. 15 (1) (2018) 344–355.
[45] V.K. Garg, Y. Narahari, M.N. Murty, Novel biobjective clustering (BiGC) based on cooperative game theory, IEEE Trans. Knowl. Data Eng. 25 (5) (2013) 1070–1082.
[46] Internet2 open science, scholarship and services exchange, URL http://www.internet2.edu/network/ose/.
[47] S. Knight, H.X. Nguyen, N. Falkner, R. Bowden, M. Roughan, The internet topology zoo, 29, 2011, pp. 1765–1775.